Diary Data Analysis

Anne Gaule

2025-08-12

Contents

1) Group numbers for analyses: 3

2) Impact of Training on Child Rated Prosocial Behaviour at Two Weeks 4

2.1) Plotting Child Prosocial Behaviour Diary Ratings at D1 & D2 4

2.2) Model Specification in Preregistration 6

2.3) Preregistered Model: Linear mixed model predicting child Prosocial behaviour at D2 by training condition, adjusted for behaviour at D1 7

2.3.1) Checking Residuals: 8

2.4) Bootstrapping Preregistered LMM: This gives singular fit error 9

2.4) Running simple regression instead 9

2.4.1) Plotting Residuals 9

2.5) Final Model: Bootstrapped Linear Regression 10

2.5.1) Results (standardised first) 11

2.5.2) Confidence Intervals 11

2.5.3) Bootstrap Diagnostics 13

3) Impact of Training on Teacher Rated Prosocial Behaviour at Two Weeks 15

3.1) Plotting teacher rated Prosocial Behaviour at D1 & D2 15

2.2) Model Specification in preregistration 17

2.3) Preregistered Linear mixed model predicting ASB at D2 by training condition, adjusted for behaviour at D1. 18

2.3.1) Investigating singular fit 18

2.4) Simplified LMM with random effect of teacher only. This does fit. 19

2.4.1) Assumption checks show serious violations 20

2.5) Simple Linear Regression 21

2.5.1) Assumption Checks (indicate LM better fit) 22

2.6) Final Model: Bootstrapped LM model 23

2.6.1) Results (standardised first) 24

2.6.2) Confidence Intervals 24
2.6.3) Bootstrap Diagnostics 26

1) Group numbers for analyses:

- Main analyses
  - Child psb = 83
  - Teach psb = 102

Table 1: Child PSB

Training_Condition	n
CONTROL	44
TRAIN	39

```
## [1] "n participants total = "
```

```
## [1] 83
```

Table 2: teach PSB

Training_Condition	n
CONTROL	53
TRAIN	49

```
## [1] "n participants total = "
```

```
## [1] 102
```

2) Impact of Training on Child Rated Prosocial Behaviour at Two Weeks

2.1) Plotting Child Prosocial Behaviour Diary Ratings at D1 & D2



Difference plot:



2.2) Model Specification in Preregistration

Model specification in SAP:

$$\text{Child_PSB}_j = (\beta_0 + u_k) + \beta_1 \text{PSB.0}_j + \beta_2 \text{Train}_j + \varepsilon_j$$

Where:

- Child_PSB_j: Child-rated antisocial behaviour at D2 for participant j.
- $j = 1, \ldots, N$ identifies the participants.
- β_0 is a fixed intercept representing the average baseline level of the dependent variable (Child_PSB_j) across all participants when all predictors are zero.
- $u_k \sim \mathcal{N}(0, \sigma_u^2)$ is the random effect for schools, accounting for variability between schools.
- $\beta_1 \text{PSB.0}_j$: effect of child-rated antisocial behaviour score at baseline (D1) on ASB at D2.
- β_2 is the fixed effect of training condition (Train; an indicator variable coded as 0 for controls or 1 for training group) on the outcome, indicating how being in the training group influences the outcome.
- $\varepsilon_j \sim \mathcal{N}(0, \sigma_\varepsilon^2)$ is random error.

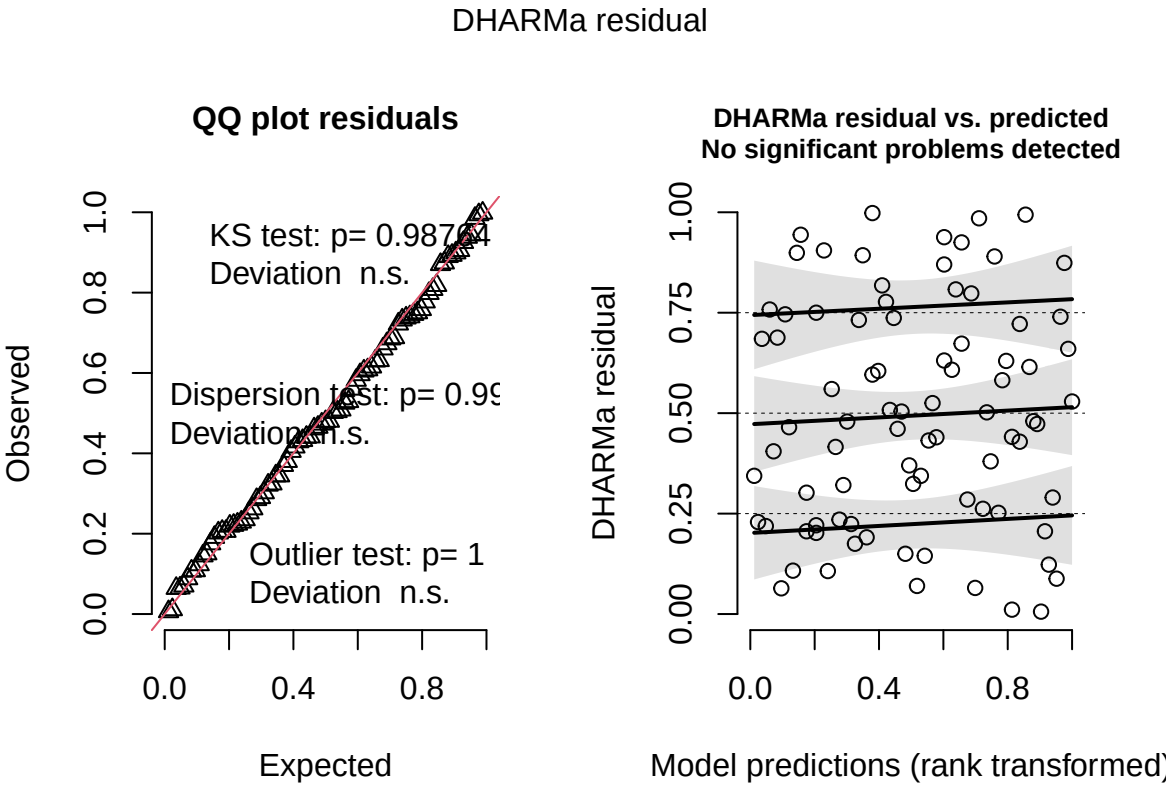
2.3) Preregistered Model: Linear mixed model predicting child Prosocial behaviour at D2 by training condition, adjusted for behaviour at D1

This model does not give a singular fit error. However, the bootstrapped version (run for consistency with other models) does.

```
model_c_psb_1 <- lmer(PSB_sum_D2 ~ PSB_sum_D1 + Training_Condition +
                      (1|School.ID),
                      data =  child_psb_d1d2_wide)
summary(model_c_psb_1)
```

```
## Linear mixed model fit by REML. t-tests use Satterthwaite's method [
## lmerModLmerTest]
## Formula: PSB_sum_D2 ~ PSB_sum_D1 + Training_Condition + (1 | School.ID)
##    Data: child_psb_d1d2_wide
##
## REML criterion at convergence: 705.6
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -2.63962 -0.64724 -0.06347  0.56513  2.62439
##
## Random effects:
##  Groups    Name        Variance Std.Dev.
##  School.ID (Intercept)   8.25    2.872
##  Residual              309.74   17.599
## Number of obs: 83, groups:  School.ID, 15
##
## Fixed effects:
##                          Estimate Std. Error      df t value Pr(>|t|)
## (Intercept)                0.2614     5.7139 70.8656   0.046    0.964
## PSB_sum_D1                 0.9873     0.0932 79.9066  10.593   <2e-16 ***
## Training_ConditionTRAIN    5.9619     3.9365 76.5588   1.514    0.134
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##             (Intr) PSB__D
## PSB_sum_D1  -0.871
## Trnn_CTRAIN -0.451  0.155
```

2.3.1) Checking Residuals:



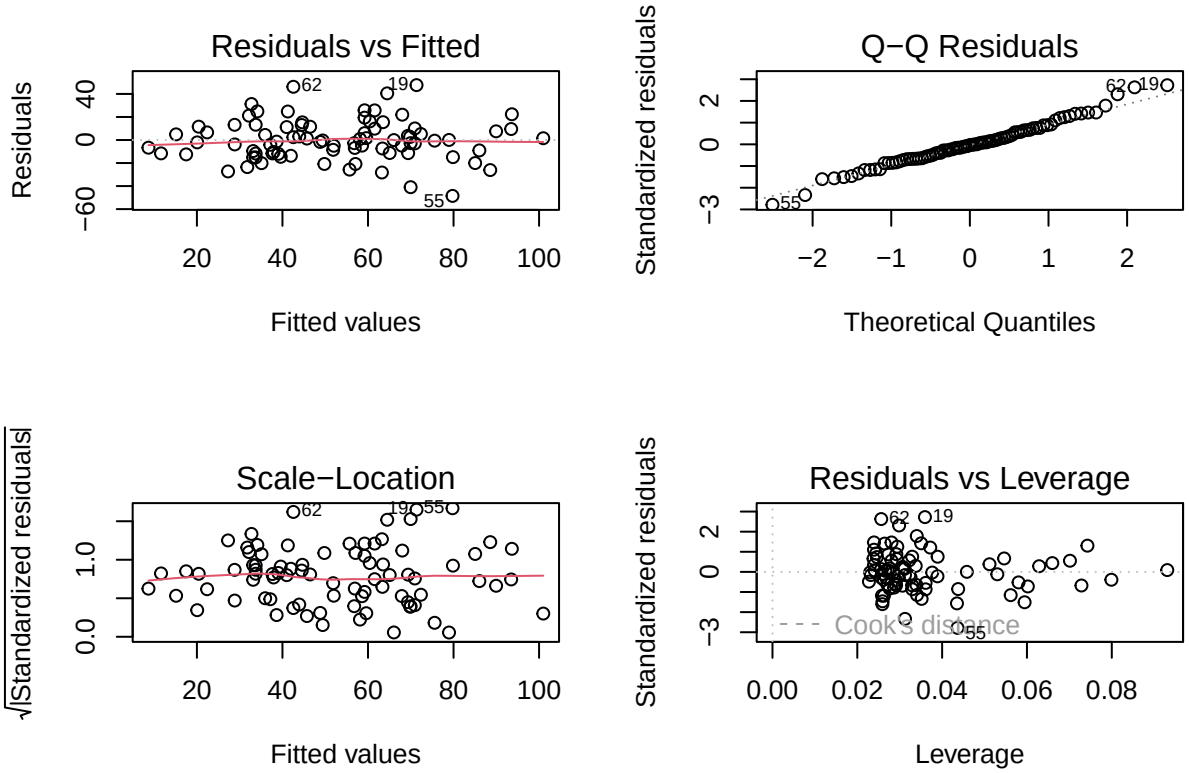
2.4) Bootstrapping Preregistered LMM: This gives singular fit error

```
# REB bootstrap advised when assumptions are violated
boot_results <- lmeresampler::bootstrap(model_c_psb_1, .f = fixef, type = "reb", B = 5000)
```

```
## Warning in reb_bootstrap.lmerMod(model, .f, B, reb_type, .refit): 'reb_type'
## unspecified, performing REB 0 bootstrap
```

2.4) Running simple regression instead

2.4.1) Plotting Residuals



2.5) Final Model: Bootstrapped Linear Regression

```
# non-parametric case bootstrap: https://journal.r-project.org/articles/RN-2002-017/RN-2002-017.pdf
boot_coef_c_psb <- function(data, indices) {
  d <- data[indices, ]  # Resample with replacement
  fit <- lm(PSB_sum_D2 ~ PSB_sum_D1 + Training_Condition,
                    data =  d)
  return(coef(fit))
}


# Run bootstrap
set.seed(123)  # For reproducibility
boot_results_c_psb <- boot(data = child_psb_d1d2_wide,
                      statistic = boot_coef_c_psb,
                      R = 5000)
```

Standardised results:

```
child_psb_d1d2_wide$Training_Condition_standardised <- standardise(child_psb_d1d2_wide$Training_Conditi
child_psb_d1d2_wide$PSB_sum_D1_standardised  <- standardise(child_psb_d1d2_wide$PSB_sum_D1)
child_psb_d1d2_wide$PSB_sum_D2_standardised  <- standardise(child_psb_d1d2_wide$PSB_sum_D2)

# Define a function that returns coefficients:
# taken from(https://www.datacamp.com/doc/r/bootstrapping)
boot_coef_c_psb_standardised <- function(data, indices) {
  d <- data[indices, ]  # Resample with replacement
  fit <- lm(PSB_sum_D2_standardised ~ PSB_sum_D1_standardised + Training_Condition_standardised,
                    data =  d)
  return(coef(fit))
}

# get standardised parameters
set.seed(12345)  # For reproducibility
boot_results_c_psb_standardised <- boot(data = child_psb_d1d2_wide,
                      statistic = boot_coef_c_psb_standardised,
                      R = 5000)
```

2.5.1) Results (standardised first)

```
##
## ORDINARY NONPARAMETRIC BOOTSTRAP
##
##
## Call:
## boot(data = child_psb_d1d2_wide, statistic = boot_coef_c_psb_standardised,
##     R = 5000)
##
##
## Bootstrap Statistics :
##       original        bias    std. error
## t1* -0.1084845  0.0016100318  0.09717782
## t2*  0.7715086 -0.0001545053  0.07142579
## t3*  0.2308772 -0.0033049642  0.14299464
```

```
##
## ORDINARY NONPARAMETRIC BOOTSTRAP
##
##
## Call:
## boot(data = child_psb_d1d2_wide, statistic = boot_coef_c_psb,
##     R = 5000)
##
##
## Bootstrap Statistics :
##      original       bias    std. error
## t1* 0.4416934  0.079490022  4.96672250
## t2* 0.9793064 -0.002067222  0.08847852
## t3* 6.2717871  0.052609594  3.89891917
```

2.5.2) Confidence Intervals

```
## BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
## Based on 5000 bootstrap replicates
##
## CALL :
## boot.ci(boot.out = boot_results_c_psb_standardised, type = "bca",
##     index = 1)
##
## Intervals :
## Level       BCa
## 95%   (-0.2994,  0.0873 )
## Calculations and Intervals on Original Scale
```

```
## BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
## Based on 5000 bootstrap replicates
##
## CALL :
## boot.ci(boot.out = boot_results_c_psb_standardised, type = "bca",
##     index = 2)
##
```

```
## Intervals :
## Level       BCa
## 95%   ( 0.6191,  0.9021 )
## Calculations and Intervals on Original Scale

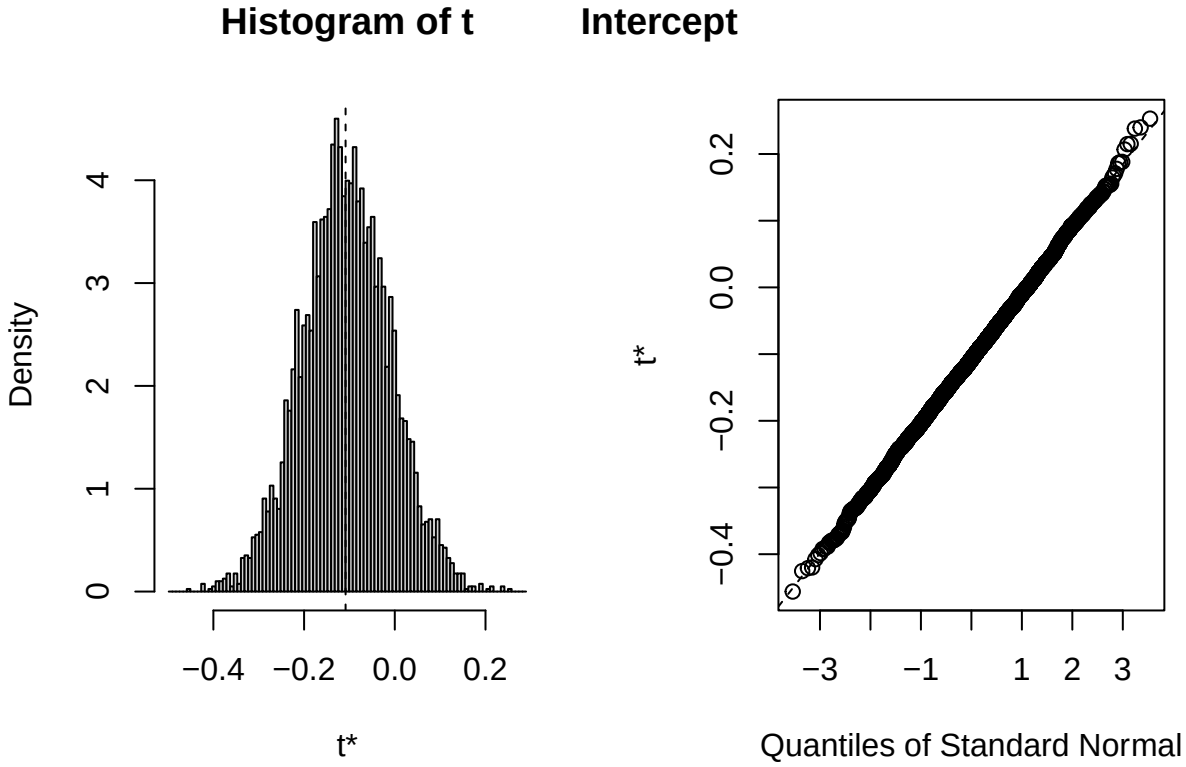
## BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
## Based on 5000 bootstrap replicates
##
## CALL :
## boot.ci(boot.out = boot_results_c_psb_standardised, type = "bca",
##     index = 3)
##
## Intervals :
## Level       BCa
## 95%   (-0.0399,  0.5205 )
## Calculations and Intervals on Original Scale

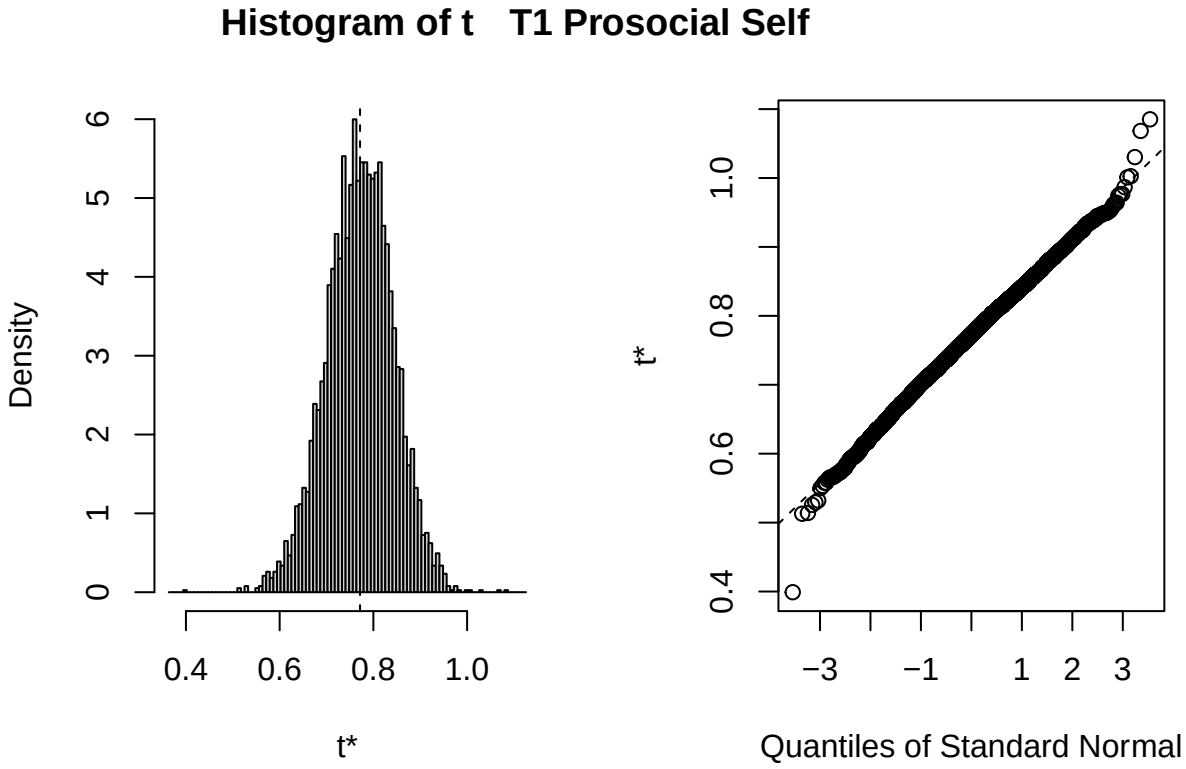
## BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
## Based on 5000 bootstrap replicates
##
## CALL :
## boot.ci(boot.out = boot_results_c_psb, type = "bca", index = 1)
##
## Intervals :
## Level       BCa
## 95%   (-8.9081, 10.8790 )
## Calculations and Intervals on Original Scale

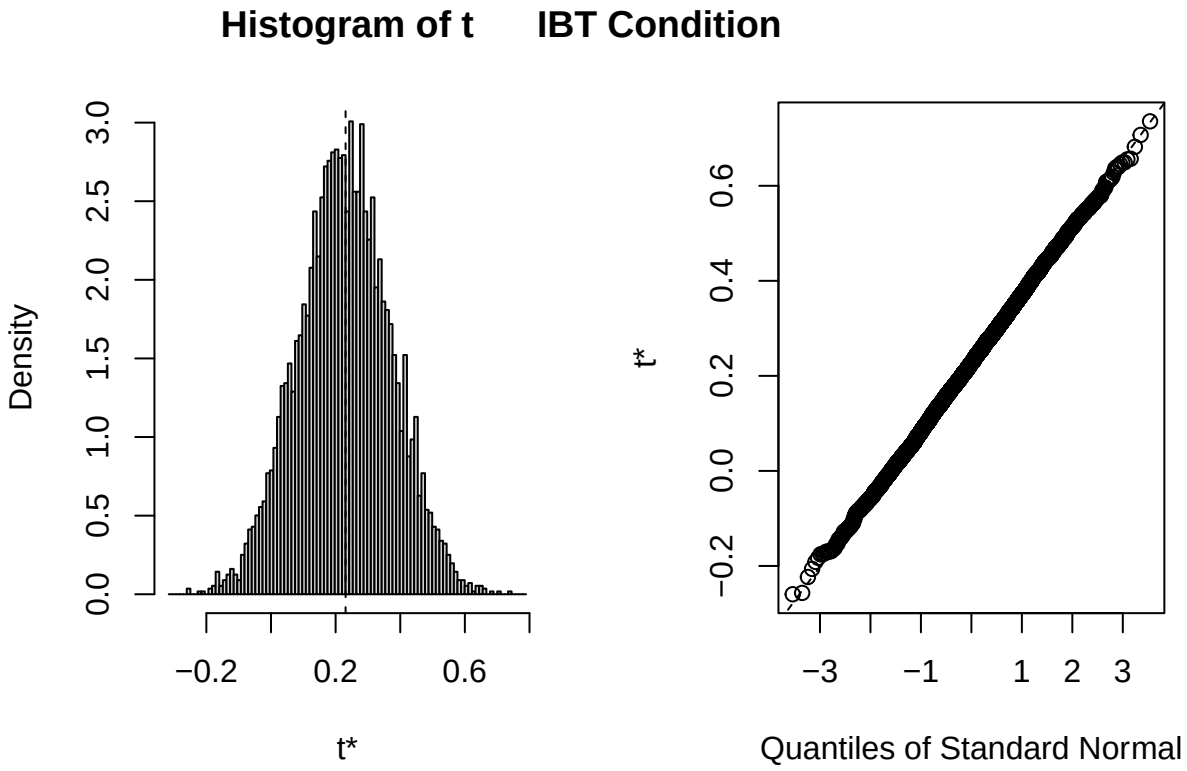
## BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
## Based on 5000 bootstrap replicates
##
## CALL :
## boot.ci(boot.out = boot_results_c_psb, type = "bca", index = 2)
##
## Intervals :
## Level       BCa
## 95%   ( 0.8011,  1.1518 )
## Calculations and Intervals on Original Scale

## BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
## Based on 5000 bootstrap replicates
##
## CALL :
## boot.ci(boot.out = boot_results_c_psb, type = "bca", index = 3)
##
## Intervals :
## Level       BCa
## 95%   (-1.250, 14.217 )
## Calculations and Intervals on Original Scale
```

2.5.3) Bootstrap Diagnostics

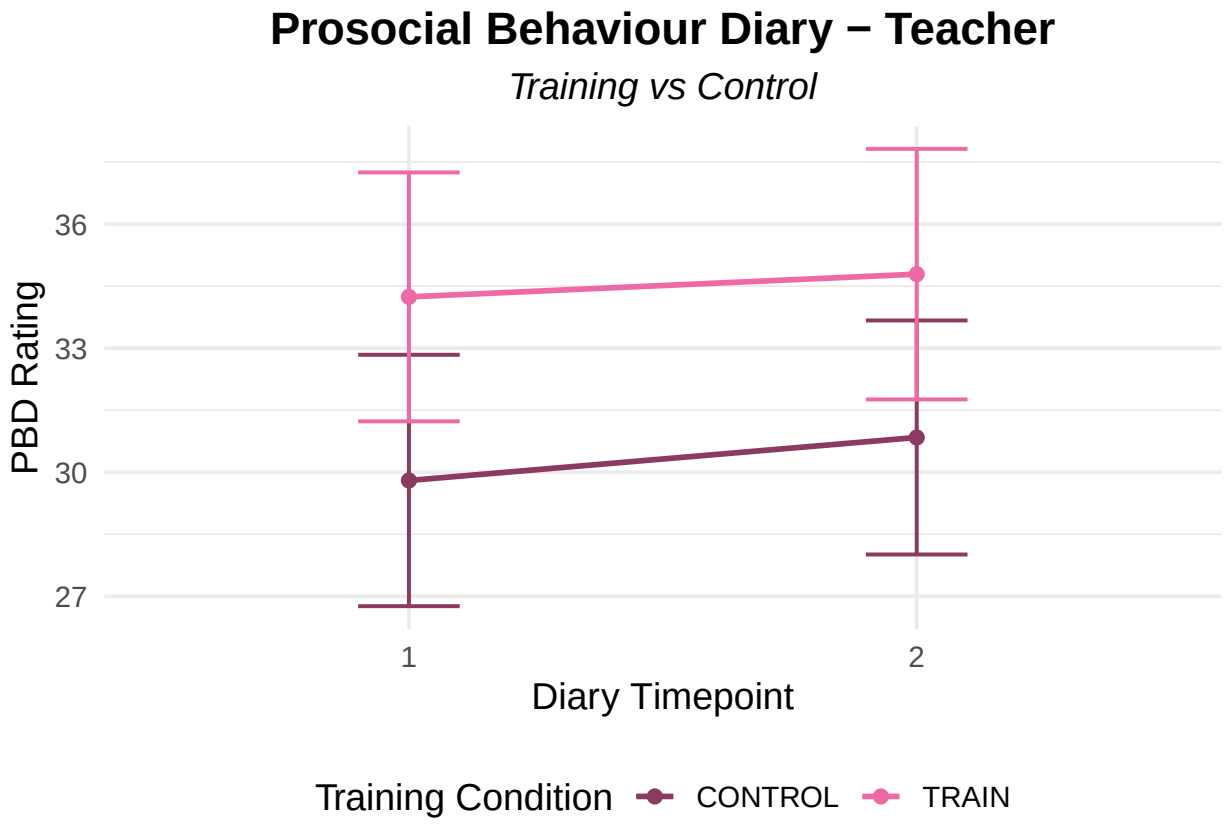





Histogram of t IBT Condition
Density
0.0
0.5
1.0
1.5
2.0
2.5
3.0
−0.2
0.2
0.6
t*
t*
−0.2
0.0
0.2
0.4
0.6
−3
−1
1
2
3
Quantiles of Standard Normal

3) Impact of Training on Teacher Rated Prosocial Behaviour at Two Weeks

3.1) Plotting teacher rated Prosocial Behaviour at D1 & D2



Difference plot:



2.2) Model Specification in preregistration

$$\text{Teach_PSB}_j = (\beta_0 + u_k + u_{i(k)}) + \beta_1 \text{PSB.0}_j + \beta_2 \text{Train}_j + \varepsilon_j$$

where:

- Teach_PSB_j: Teacher-rated prosocial behaviour (at D2) for participant j.
- $j = 1, \ldots, N$ identifies the participants.
- $u_k \sim \mathcal{N}(0, \sigma_u^2)$ is the random effect for schools, accounting for variability between schools.
- $u_{i(k)} \sim \mathcal{N}(0, \sigma_u^2)$ is the random effect for teachers, nested within schools.
- β_0 is a fixed intercept representing the average baseline level of the dependent variable (Teach_ASB_j) across all participants when all predictors are zero.
- $\beta_1 \text{PSB.0}_j$: effect of teacher-rated antisocial behaviour score at baseline (D1) on later prosocial behaviour.
- β_2 is the fixed effect of Training condition (Train; an indicator variable coded as 0 for controls or 1 for training group) on the outcome, indicating how being in the training group influences the outcome.
- $\varepsilon_j \sim \mathcal{N}(0, \sigma_\varepsilon^2)$ is the random error term.

2.3) Preregistered Linear mixed model predicting ASB at D2 by training condition, adjusted for behaviour at D1.

Once again model has a singular fit. This time it seems to be school that is not contributing to the model.

```
model_t_psb_1 <- lmer(PSB_mean_D2~ PSB_mean_D1 + Training_Condition +
                      (1|School.ID/teachUUID_diaries),
                      data =  teach_psb_d1d2_wide)
```

```
## boundary (singular) fit: see help('isSingular')
```

2.3.1) Investigating singular fit

```
##  Groups                       Name        Std.Dev.
##  teachUUID_diaries:School.ID (Intercept)  8.5374
##  School.ID                   (Intercept)  0.0000
##  Residual                                 12.8362
```

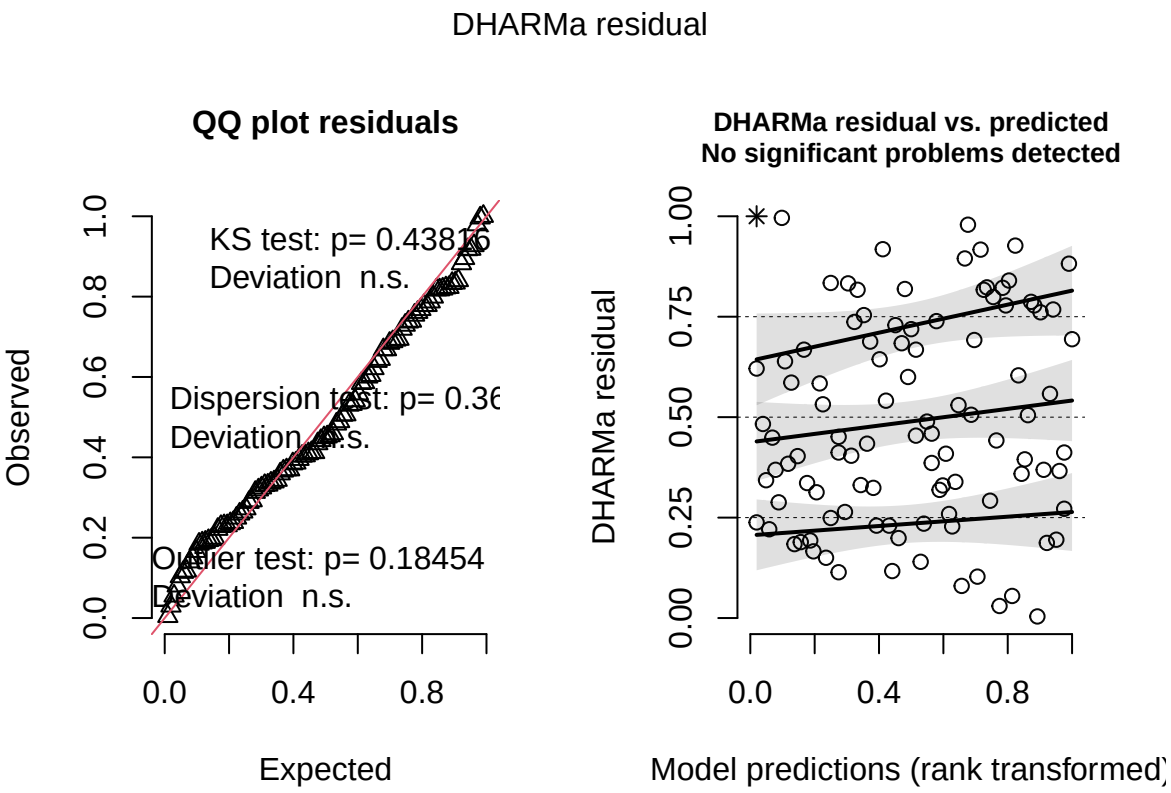
2.4) Simplified LMM with random effect of teacher only. This does fit.

```
model_t_psb_2 <- lmer(PSB_mean_D2~ PSB_mean_D1 + Training_Condition +
                      (1|teachUUID_diaries),
                      data =  teach_psb_d1d2_wide)

summary(model_t_psb_2)
```

```
## Linear mixed model fit by REML. t-tests use Satterthwaite's method [
## lmerModLmerTest]
## Formula:
## PSB_mean_D2 ~ PSB_mean_D1 + Training_Condition + (1 | teachUUID_diaries)
##    Data: teach_psb_d1d2_wide
##
## REML criterion at convergence: 830.3
##
## Scaled residuals:
##     Min      1Q  Median      3Q     Max
## -2.5397 -0.5454 -0.1350  0.5934  3.7000
##
## Random effects:
##  Groups            Name        Variance Std.Dev.
##  teachUUID_diaries (Intercept)  76.25    8.732
##  Residual                      163.34   12.780
## Number of obs: 102, groups:  teachUUID_diaries, 36
##
## Fixed effects:
##                          Estimate Std. Error       df t value Pr(>|t|)
## (Intercept)              12.83639    3.09559 80.80741   4.147 8.27e-05 ***
## PSB_mean_D1               0.56481    0.07311 98.18077   7.725 9.64e-12 ***
## Training_ConditionTRAIN  2.09281    2.85929 93.31834   0.732    0.466
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##             (Intr) PSB__D
## PSB_mean_D1 -0.622
## Trnn_CTRAIN -0.304 -0.184
```

2.4.1) Assumption checks show serious violations



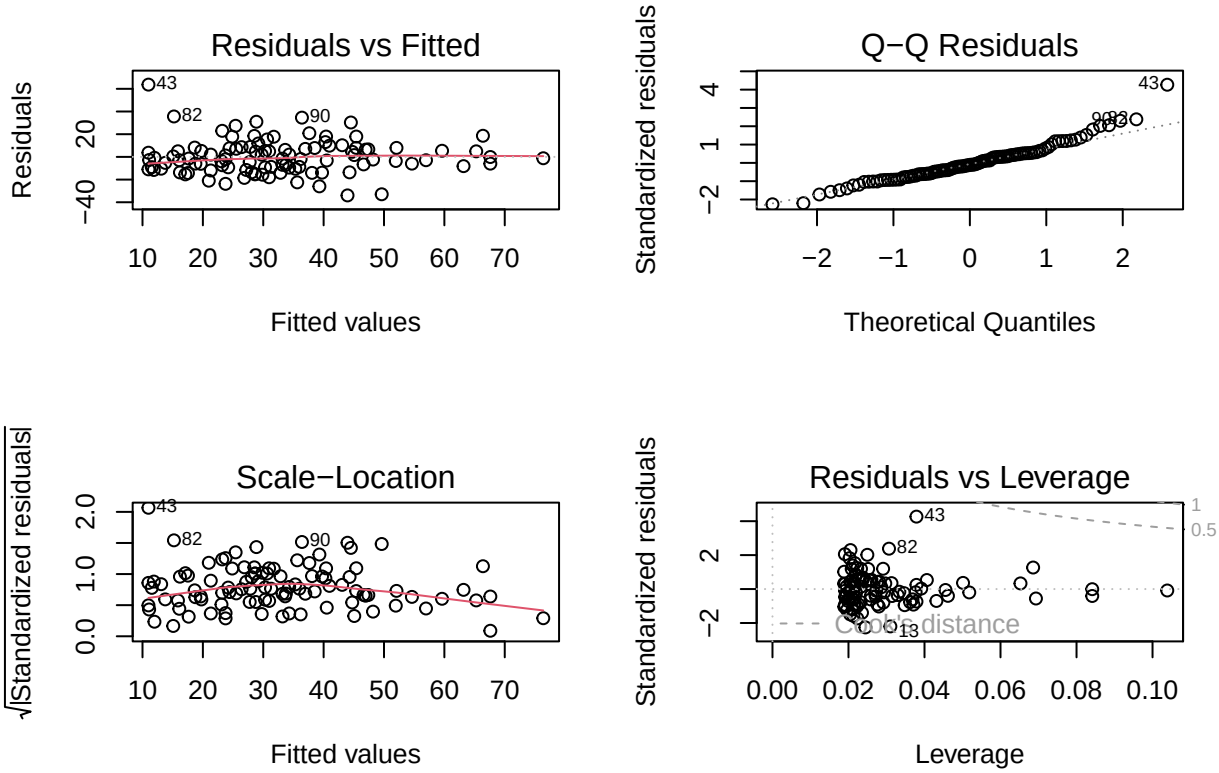
2.5) Simple Linear Regression

```
model_t_psb_2_lm <- lm(PSB_mean_D2~ PSB_mean_D1 + Training_Condition ,
                       data =  teach_psb_d1d2_wide)

summary(model_t_psb_2_lm)
```

```
##
## Call:
## lm(formula = PSB_mean_D2 ~ PSB_mean_D1 + Training_Condition,
##     data = teach_psb_d1d2_wide)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -33.958  -9.424  -1.693   7.472  63.679
##
## Coefficients:
##                           Estimate Std. Error t value Pr(>|t|)
## (Intercept)              10.98796    2.96116   3.711 0.000341 ***
## PSB_mean_D1               0.66619    0.07041   9.462 1.63e-15 ***
## Training_ConditionTRAIN   0.99317    3.03112   0.328 0.743862
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 15.21 on 99 degrees of freedom
## Multiple R-squared:  0.4796, Adjusted R-squared:  0.4691
## F-statistic: 45.62 on 2 and 99 DF,  p-value: 9.068e-15
```

2.5.1) Assumption Checks (indicate LM better fit)



2.6) Final Model: Bootstrapped LM model

```
# non-parametric case bootstrap: https://journal.r-project.org/articles/RN-2002-017/RN-2002-017.pdf
# argued to be better  for heteroskedastic data:
boot_coef_t_psb <- function(data, indices) {
  d <- data[indices, ]  # Resample with replacement
  fit <- lm(PSB_mean_D2 ~ PSB_mean_D1 + Training_Condition,
                    data =  d)
  return(coef(fit))
}

# Run bootstrap
boot_results_t_psb <- boot(data = teach_psb_d1d2_wide,
                     statistic = boot_coef_t_psb,
                     R = 5000)
```

Standardised results:

```
teach_psb_d1d2_wide$Training_Condition_standardised <- standardise(teach_psb_d1d2_wide$Training_Conditi
teach_psb_d1d2_wide$PSB_mean_D1_standardised  <- standardise(teach_psb_d1d2_wide$PSB_mean_D1)
teach_psb_d1d2_wide$PSB_mean_D2_standardised  <- standardise(teach_psb_d1d2_wide$PSB_mean_D2)
teach_psb_d1d2_wide$PSB_mean_D3_standardised  <- standardise(teach_psb_d1d2_wide$PSB_mean_D3)

# Define a function that returns coefficients:
# taken from(https://www.datacamp.com/doc/r/bootstrapping)
boot_coef_t_psb_standardised <- function(data, indices) {
  d <- data[indices, ]  # Resample with replacement
  fit <- lm(PSB_mean_D2_standardised ~ PSB_mean_D1_standardised + Training_Condition_standardised,
                    data =  d)
  return(coef(fit))
}

# get standardised parameters
set.seed(12345)  # For reproducibility
boot_results_t_psb_standardised <- boot(data = teach_psb_d1d2_wide,
                     statistic = boot_coef_t_psb_standardised,
                     R = 5000)
```

2.6.1) Results (standardised first)

```
##
## ORDINARY NONPARAMETRIC BOOTSTRAP
##
##
## Call:
## boot(data = teach_psb_d1d2_wide, statistic = boot_coef_t_psb_standardised,
##     R = 5000)
##
##
## Bootstrap Statistics :
##        original        bias    std. error
## t1* -0.02285098 -0.0007498384   0.1022784
## t2*  0.68968024  0.0007038124   0.0668263
## t3*  0.04756736  0.0000134001   0.1391947
```

```
##
## ORDINARY NONPARAMETRIC BOOTSTRAP
##
##
## Call:
## boot(data = teach_psb_d1d2_wide, statistic = boot_coef_t_psb,
##     R = 5000)
##
##
## Bootstrap Statistics :
##       original        bias    std. error
## t1* 10.9879561 -0.0590547164  3.52462131
## t2*  0.6661914  0.0006256679  0.06563598
## t3*  0.9931694  0.0736336490  2.83699576
```

2.6.2) Confidence Intervals

```
## BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
## Based on 5000 bootstrap replicates
##
## CALL :
## boot.ci(boot.out = boot_results_t_psb_standardised, type = "bca",
##     index = 1)
##
## Intervals :
## Level       BCa
## 95%   (-0.1978,  0.2151 )
## Calculations and Intervals on Original Scale
```

```
## BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
## Based on 5000 bootstrap replicates
##
## CALL :
## boot.ci(boot.out = boot_results_t_psb_standardised, type = "bca",
##     index = 2)
##
```

```
## Intervals :
## Level       BCa
## 95%   ( 0.5335,  0.8033 )
## Calculations and Intervals on Original Scale

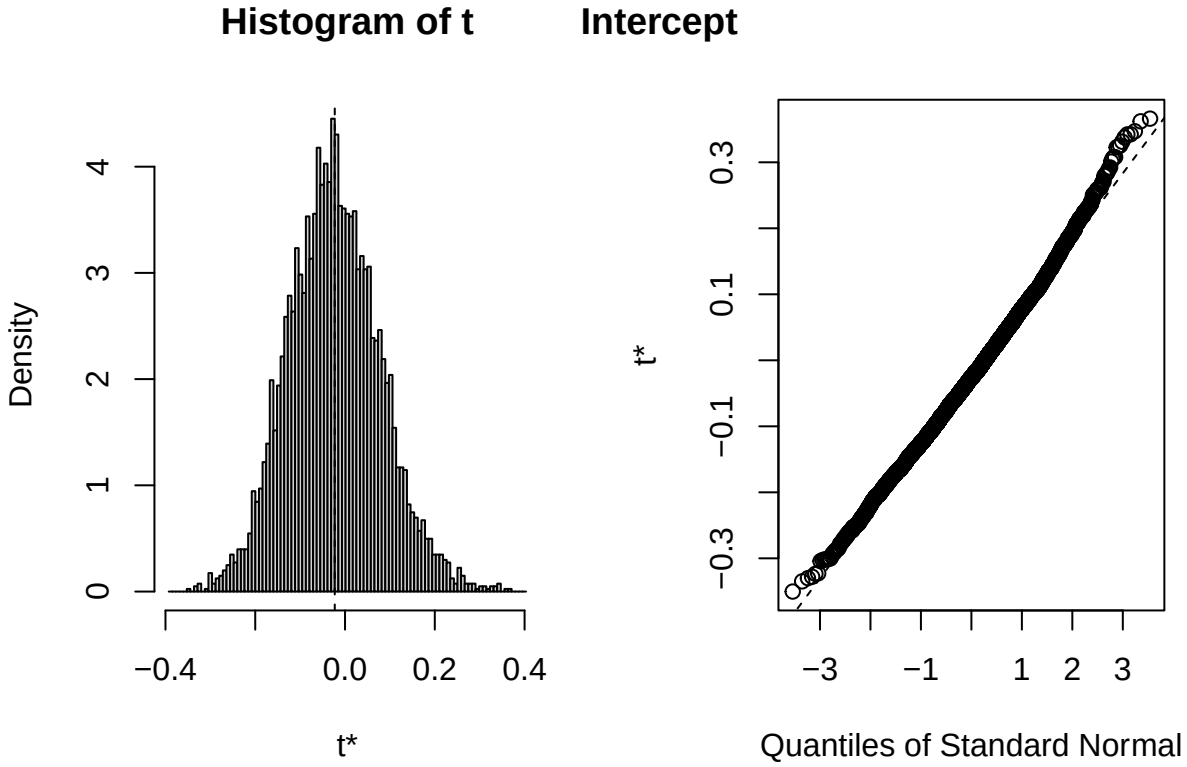
## BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
## Based on 5000 bootstrap replicates
##
## CALL :
## boot.ci(boot.out = boot_results_t_psb_standardised, type = "bca",
##     index = 3)
##
## Intervals :
## Level       BCa
## 95%   (-0.2340,  0.3141 )
## Calculations and Intervals on Original Scale

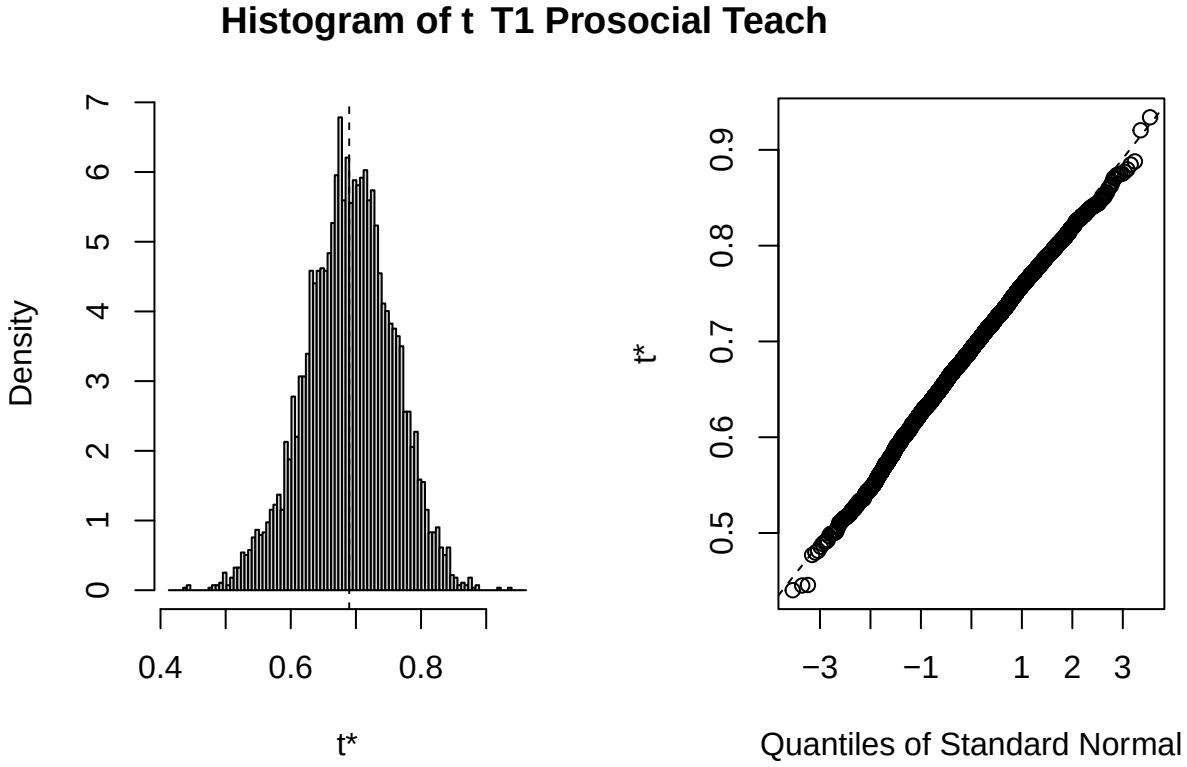
## BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
## Based on 5000 bootstrap replicates
##
## CALL :
## boot.ci(boot.out = boot_results_t_psb, type = "bca", index = 1)
##
## Intervals :
## Level       BCa
## 95%   ( 5.50, 20.31 )
## Calculations and Intervals on Original Scale

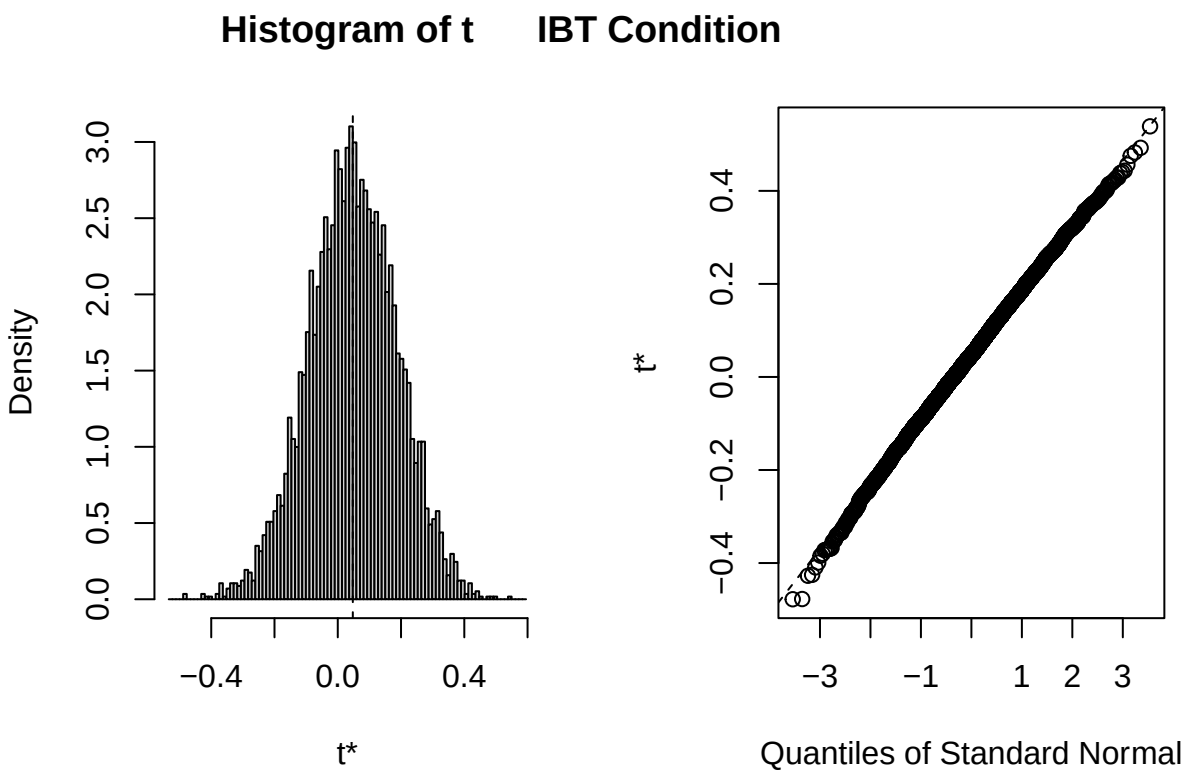
## BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
## Based on 5000 bootstrap replicates
##
## CALL :
## boot.ci(boot.out = boot_results_t_psb, type = "bca", index = 2)
##
## Intervals :
## Level       BCa
## 95%   ( 0.5024,  0.7717 )
## Calculations and Intervals on Original Scale

## BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
## Based on 5000 bootstrap replicates
##
## CALL :
## boot.ci(boot.out = boot_results_t_psb, type = "bca", index = 3)
##
## Intervals :
## Level       BCa
## 95%   (-4.7457,  6.3400 )
## Calculations and Intervals on Original Scale
```

2.6.3) Bootstrap Diagnostics






Histogram of t IBT Condition
Density
0.0
0.5
1.0
1.5
2.0
2.5
3.0
−0.4
0.0
0.4
t*
t*
−0.4
−0.2
0.0
0.2
0.4
−3
−1
1
2
3
Quantiles of Standard Normal